ENGINEERING REFERENCE / CLOUD APPLICATIONS

The Twelve-Factor App

Portable code. Explicit configuration. Replaceable processes. Predictable releases.

01 Codebase

One codebase in version control; many deploys.
Deploy the same application repository to dev, staging and production. Each independently deployable app has its own codebase; share code as libraries.
Avoid: Separate repositories per environment; untracked production edits.

02 Dependencies

Explicitly declare and isolate dependencies.
Use a manifest plus lockfile where supported, and an isolated runtime. Declare required system tools as well as language packages.
Avoid: Relying on globally installed packages or server-specific tooling.

03 Config

Store deployment configuration in the environment.
Inject credentials, service URLs and deploy-specific settings at runtime. Use granular variables; secret managers can supply sensitive values.
Avoid: Secrets in Git; large environment-specific config bundles.

04 Backing services

Treat backing services as attached resources.
Address databases, queues, caches and mail services through configuration. Swap compatible resources without changing application code.
Avoid: Hard-coded endpoints; treating a local database as special.

05 Build, release, run

Strictly separate build, release and run stages.
Build an artifact; combine it with config into an identified release; run that release. Preserve releases for repeatable deploys and rollback.
Avoid: Compiling or editing application code inside a live deployment.

06 Processes

Run stateless, share-nothing processes.
Persist durable data in backing services. Use local disk and memory only for temporary work; store sessions outside individual processes.
Avoid: Sticky in-memory sessions; user uploads on ephemeral disk.

07 Port binding

Export services through port binding.
The app binds a port with its runtime or bundled server. A platform or reverse proxy routes requests to it; use the assigned port if provided.
Avoid: Depending on a manually configured external application server.

08 Concurrency

Scale out through the process model.
Define process types, such as web and worker. Add instances of each independently. Let the execution environment supervise processes.
Avoid: One giant process that couples web traffic and background jobs.

09 Disposability

Start quickly and shut down gracefully.
On termination, stop accepting work and drain within the grace period. Finish or release jobs safely; design retries to avoid duplicate effects.
Avoid: Assuming processes never die; losing work on abrupt shutdown.

10 Dev/prod parity

Keep environments as similar as practical.
Minimize gaps in time, people and tools. Match backing-service types and versions; deploy small changes frequently through consistent workflows.
Avoid: A different local database engine masking production behavior.

11 Logs

Treat logs as event streams.
Write events to stdout; the environment collects, routes and stores them. Structured fields and correlation IDs improve diagnosis.
Avoid: App-managed rotating log files; credentials or tokens in logs.

12 Admin processes

Run admin tasks as one-off processes.
Run migrations, consoles and maintenance with the application release, config and dependencies. Version admin code alongside app code.
Avoid: Ad hoc scripts with different credentials or dependency versions.

RELEASE FLOW Source + declared dependencies → Build artifact → Artifact + config = release → Run process types
QUICK CHECK Fresh checkout builds? • Secrets external? • Restart loses no durable data? • Scale without shared local state? • Shutdown safe?

Source: 12factor.net • Original methodology by Heroku engineers • Practical examples are implementation guidance.
Scope: deployment and operations principles; complement with security, backups, metrics/traces, and safe schema migration design.

Downloaded from www.guntaka.com